

Architecting long horizon self healing Agentic AI systems for Institutional Finance

by Rajat Mehta

Principal AI Engineer

<https://www.linkedin.com/in/purushottam11/>

rajat.me@gmail.com

This talk is being given in individual capacity and is independent of my employer.

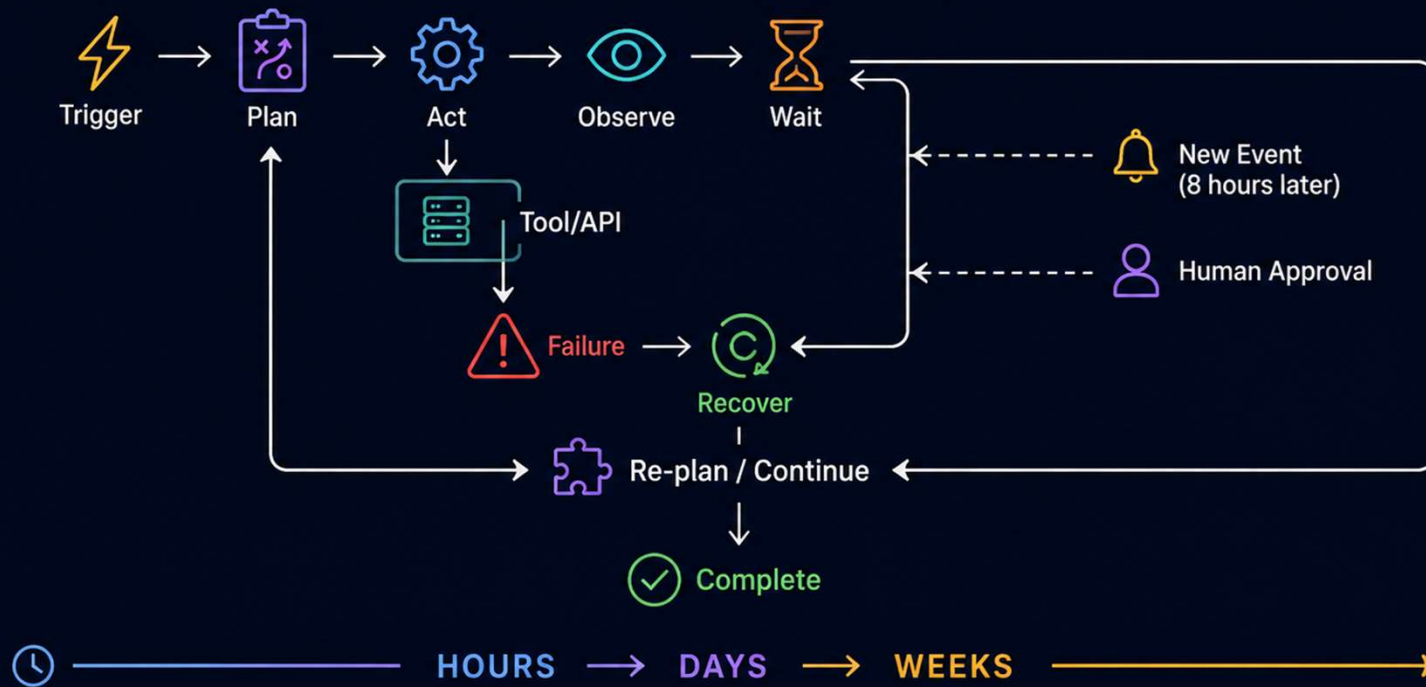
Any view, opinions, or examples presented are my own and should not be attributed to my employer

When an Agent outlives the session.

TODAY'S TYPICAL AGENT



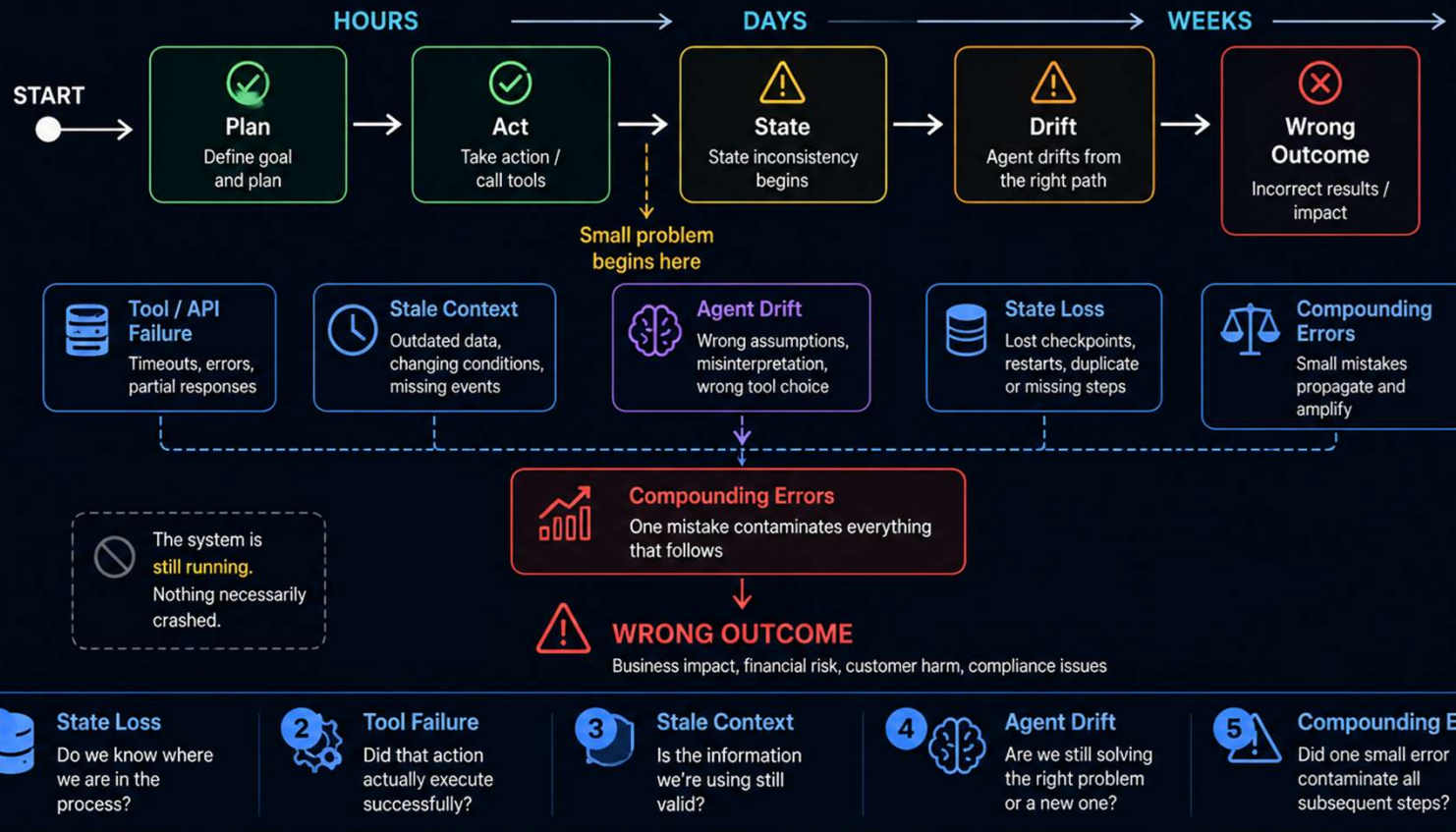
LONG-HORIZON AGENT



Why long horizon Agents are hard.

3 Why Long-Horizon Agents Are Hard

Small failures compound over time.



The dangerous agent isn't always the one that crashes.
It's the one that keeps running incorrectly.

Why Finance raises the reliability bar

4

Why Finance Raises the Reliability Bar

Institutional finance workflows are long-running, high-impact, and heavily regulated.



1. Long-lived workflows

Trades, reconciliations, investigations, approvals and regulatory reporting can span hours to days.

→ Many steps, many handoffs, many waits.



2. High cost of silent errors

A workflow can technically “succeed” while producing the wrong decision, action or report.

→ Impact can be financial, operational, reputational and regulatory.



3. Many systems & dependencies

Agents interact with multiple APIs, databases, market/reference data, legacy platforms, queues and human workflows.

→ More points of failure.



4. State changes underneath you

Prices move, positions change, limits update, approvals expire, reference data is corrected—while the agent is waiting.

→ Yesterday's truth may be invalid today.



5. Regulation & auditability

You must reconstruct what happened, why, using what data, which model, which version, and who approved it.

→ End-to-end audit trail is mandatory.



6. Humans cannot babysit every step

At scale, agents must recover automatically from routine failures and drift, and escalate only meaningful exceptions.

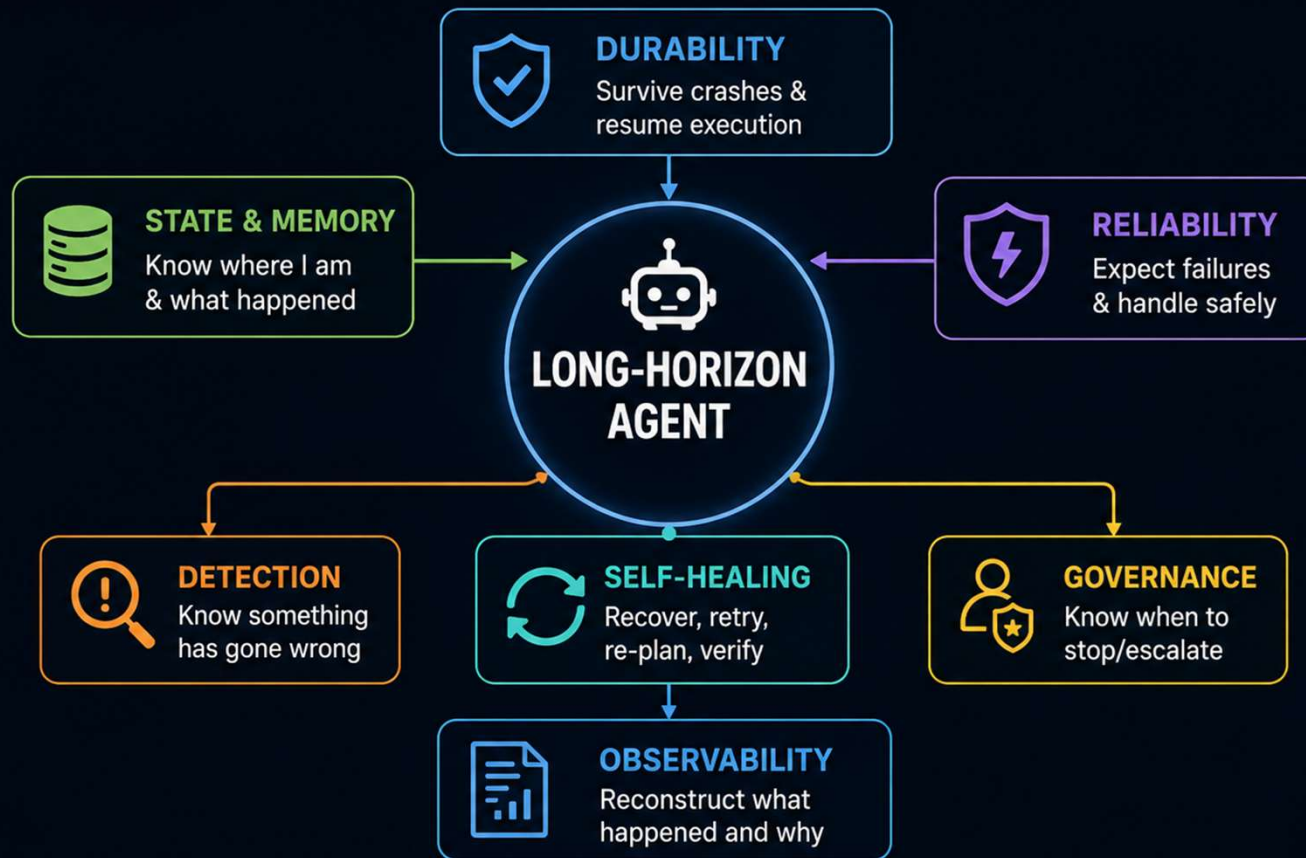
→ Otherwise, you don't get scale or trust.



In finance, an agent that runs for days isn't enough.
It must remain **correct**, **recoverable**, **auditable** and **controllable** for days.

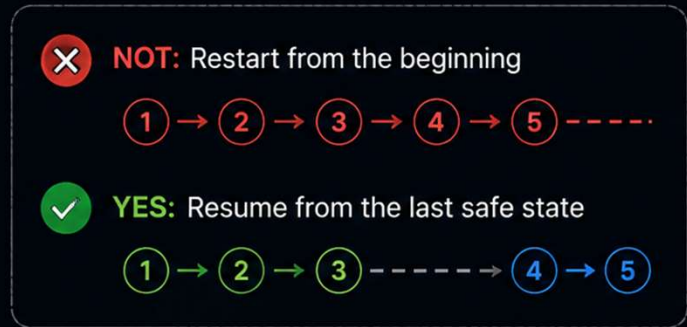
What Does a Production Long-Horizon Agent Need?

Running for days is easy. Remaining correct for days is hard.

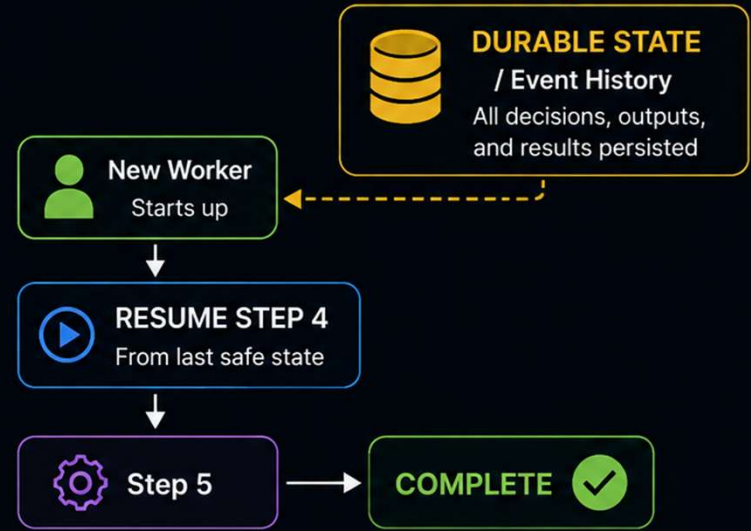


Production-ready = Survive + Remember + Detect + Recover + Explain

Durability



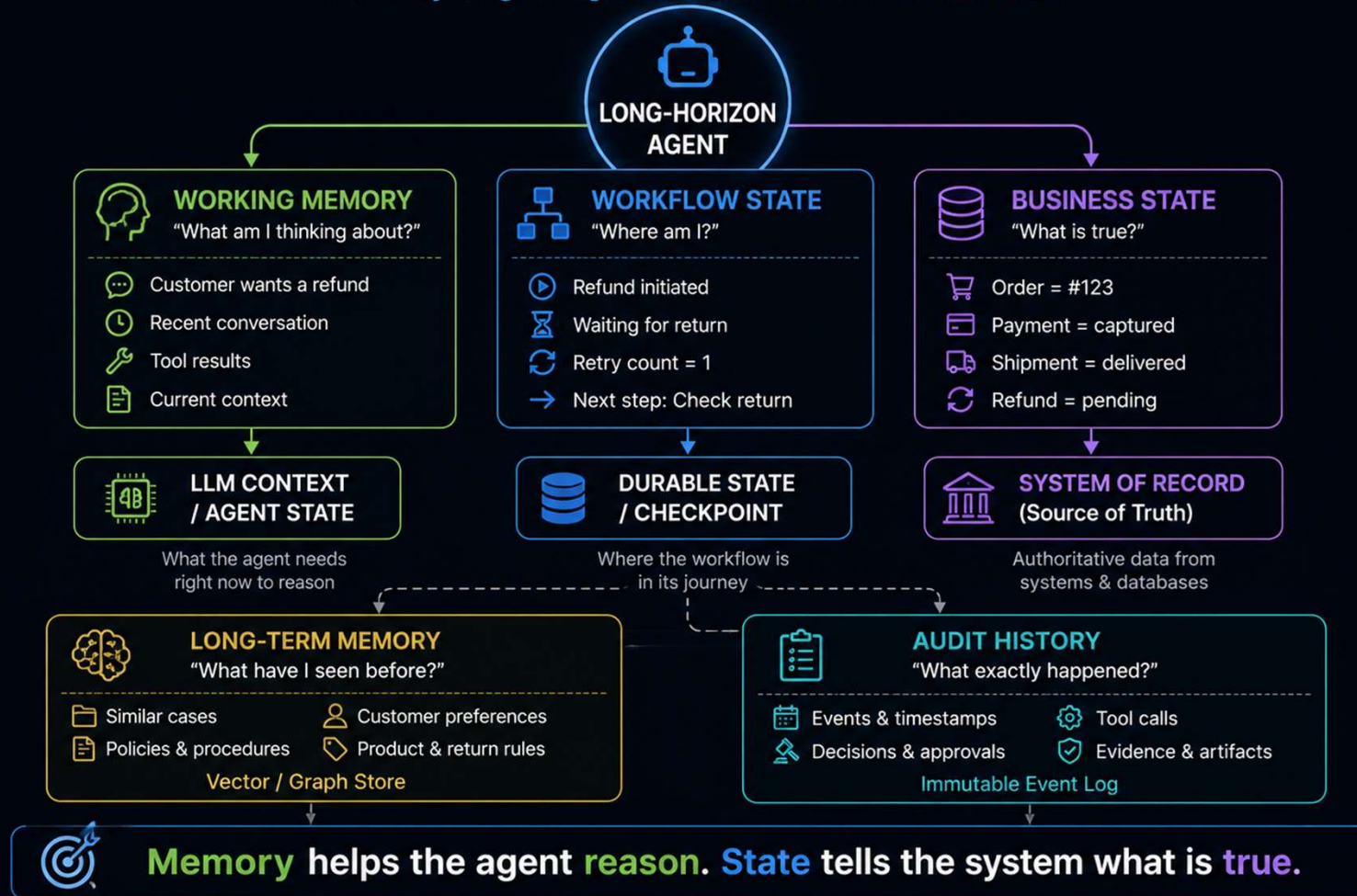
The process can die.
The workflow must not.



 Persist State Store every event, decision & result	 Replay / Resume Reconstruct state and continue	 Continue Safely No duplication, no loss, exactly once effects
--	--	---

State & Memory: Remember the Right Things

Not everything the agent remembers is authoritative truth.



Reliability: Assume Everything Will Fail

Design for failure. Recover automatically. Escalate only when needed.



The goal is not zero failures. The goal is zero unhandled failures.

EXAMPLES IN ACTION



Place Order
Payment API fails →
retry → success
Order placed



Track Shipment
Tracking service down →
fallback to alternate API →
Status returned



Send Email
SMTP timeout →
retry → fallback provider →
Email delivered



Process Refund
Refund fails after capture →
compensate (reverse auth) →
Customer not charged



Update Inventory
DB write fails →
retry → success
Inventory updated



Assume everything will fail. **Detect** quickly. **Recover** automatically. **Verify** always. **Escalate** wisely.

Detection: The Hard Part — **Explicit** vs **Silent** Failures

If you can't detect it, you can't fix it.



Agent Drift & Compounding Errors

Small mistakes compound. Drift turns wrong into very wrong.

✓ INTENDED PATH



Why It Happens

- Incorrect assumptions
- Outdated context
- Bad tool results
- Missing validations
- Ambiguous instructions

✗ WITH DRIFT & COMPOUNDING ERRORS



COMPOUNDING IMPACT



Wrong orders shipped



Refunds & replacements



Customer trust lost



More time & rework



Cost & operational noise



Reputation at risk



Drift is silent. By the time it's visible, damage is already done.

Self-Healing

Self-Healing: Detect → Diagnose → Recover → Verify

Close the loop. Don't just retry blindly.



Self-healing turns failures into **non-events**.

Recovery Patterns Choose the Right Tool for the Situation

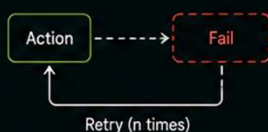
💡 No single pattern works everywhere. Good systems use many.

1. RETRY



Try the same action again.
Best for transient failures.

How it works



Example (E-commerce)
Payment gateway times out.
Retry the charge.

2. CHECKPOINT



Save progress so we can
safely resume later.

How it works



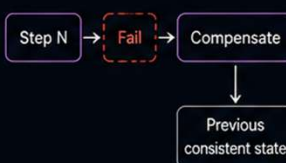
Example (E-commerce)
Order created and items reserved.
If the workflow restarts, resume
from "Payment" step.

3. COMPENSATION



Undo or offset the effect of
a failed step.

How it works



Example (E-commerce)
Payment captured but order
creation failed.
Issue refund to undo charge.

4. RE-PLAN



Create a new plan when
assumptions or context change.

How it works



Example (E-commerce)
Item out of stock.
Re-plan: suggest alternate
product or cancel item.

5. FALLBACK



Use an alternate tool,
model, or path.

How it works



Example (E-commerce)
Primary fraud service down.
Use backup fraud service
or rules engine.



**When to Use
What?**

- **Retry:** Transient errors (timeouts, 5xx)
- **Checkpoint:** Long workflows, restarts

- **Compensation:** Partial failure after side-effects
- **Re-plan:** Assumptions changed, new info

- **Fallback:** Primary system unavailable



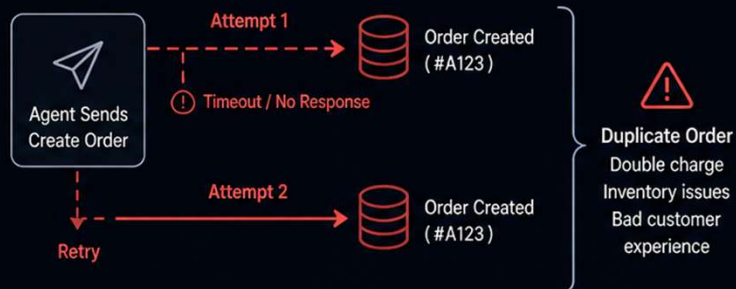
Goal: Keep making progress safely until the work is done — or escalate with full context.

Idempotency & Exactly-Once Effects

Make retries safe. One logical action → One real-world effect.

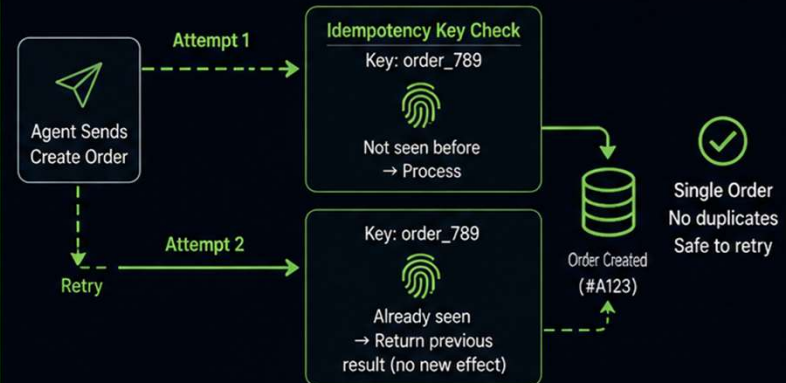
✗ Without Idempotency

Retries create duplicate side effects.



✓ With Idempotency

Retries are safe. Only one effect.



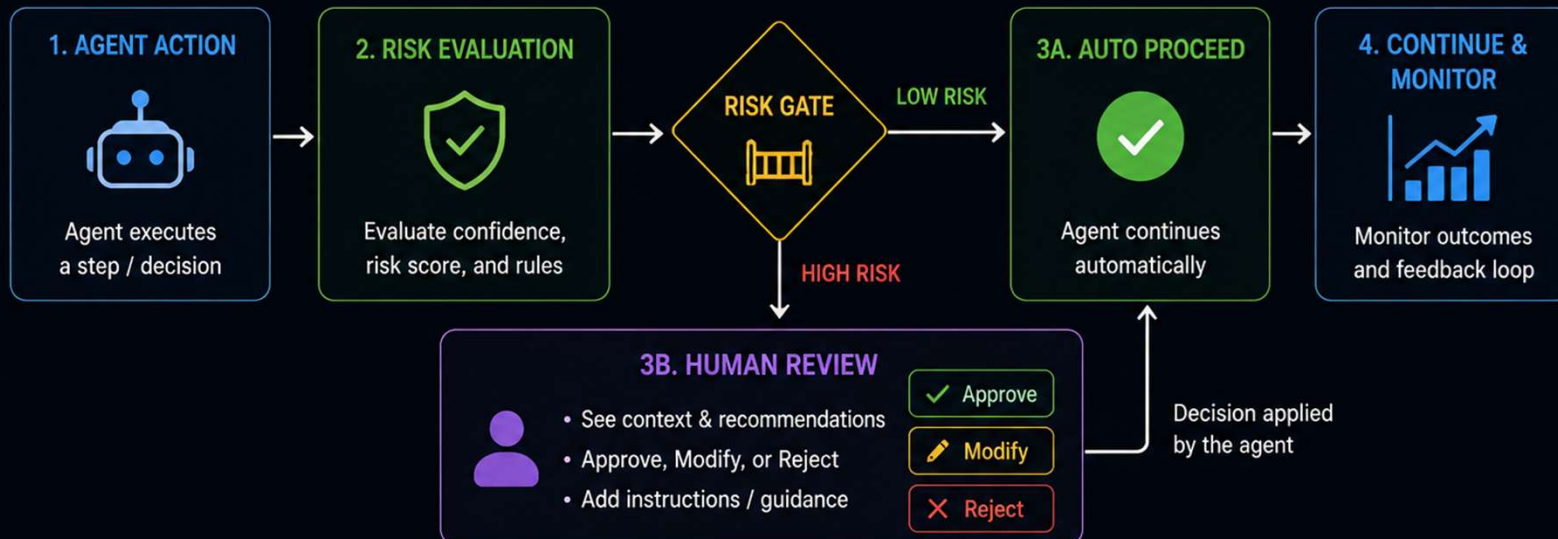
How It Works

- 1 Agent generates a unique Idempotency Key per logical action
- 2 Key is sent with the request
- 3 System checks if the key was already processed
- 4 If yes → return the previous result (no new effect)
- 5 If no → process the action, store result with the key, and return response

★ **Goal:** Turn retries from a risk into a **safety net**.

Human-in-the-Loop / Risk Gates

Automate fast. Escalate smart. Keep **humans in control** for what matters.



When to Escalate

- Low confidence / high uncertainty
- High financial or legal impact
- Policy / compliance checks
- New or unseen scenarios



Examples

- Large payments / wire transfers
- New vendors / counterparties
- Contract changes / exceptions
- Regulatory or audit actions



Benefits

- Better decisions on high-risk items
- Fewer errors and surprises
- Auditability and trust
- Continuous learning from feedback

★ **Goal:** Right balance of automation and human judgment. **Speed with trust.**

Observability for Long-Running Agentic Systems

Make behavior visible. Understand what's happening. Improve continuously.



Metrics

- System health and performance
- Throughput, latency, error rate
 - Resource usage (CPU, memory)
 - Queue depth, backlog
 - Cost and token usage



Logs

- What happened and why
- Structured application logs
 - Error and exception details
 - Business events and outcomes
 - Audit and compliance logs



Traces

- End-to-end request flow
- Agent run → steps → tools
 - Latency at each step
 - Dependencies and calls
 - Spans with timing & status



Events

- State changes and signals
- Run started / completed
 - Step success / failure
 - Retries, timeouts, recoveries
 - Alerts and notifications



Outcomes

- Business impact
- Success rate of workflows
 - SLOs / SLIs
 - Quality of answers / actions
 - Customer impact and value

HOW WE OBSERVE



Instrument

Add instrumentation in agents, tools and workflows.



Collect

Gather telemetry from all components and external services.



Store

Store in systems built for metrics, logs, traces and events.



Analyze

Query, correlate and explore to understand behavior and issues.



Alert & Act

Detect issues early, notify the right people and trigger automation.



Improve

Learn from signals, fix root causes and optimize continuously.

EXAMPLES OF QUESTIONS WE CAN ANSWER

- ? Why did a workflow fail?
- ? Which step is the slowest?
- ? Are error rates rising?
- ? Are retries increasing?
- ? Which tools are most failing?
- ? Is response quality degrading?
- ? What is the impact on users?
- ? Are we meeting our SLOs?

EXAMPLES OF TOOLS (CATEGORIES)



Metrics

- Time series DB
- Metrics collectors
- Dashboarding



Logs

- Log aggregation
- Search & indexing
- Log analytics



Traces

- Distributed tracing
- Trace storage
- Service maps



Alerts

- Alerting engine
- Notification channels
- Incident management



Dashboards

- Visualization
- Correlation
- Drill-down

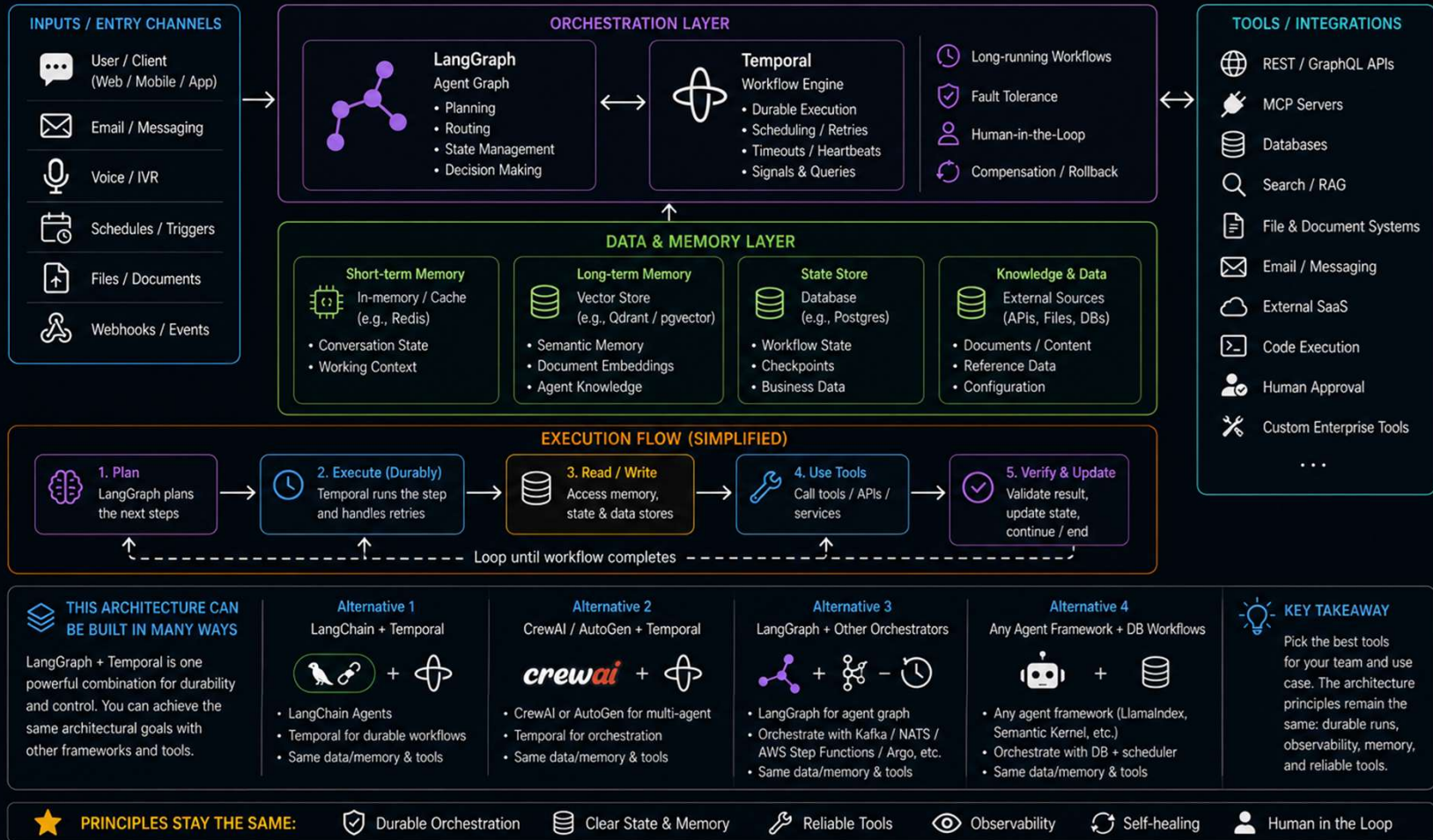


Goal: End-to-end visibility across agents, steps, tools and systems to deliver reliable, trusted outcomes.

Reference Architecture

Reference Architecture — LangGraph + Temporal + Data/Memory + Tools

A durable, observable, and self-healing foundation for long-running agentic systems



DEMO

Ecommerce Domain – Damaged product Refund Request through an AI Agent workflow

Case A-7741. Maya ordered a **\$189 coffee machine**. It arrived damaged. Maya wants a full refund. **Let's see how the agent handles the case end to end.**

Why it is Hard ?

It looks like a form, it isn't. Look at the full agent journey

CASE A-7741 · WHAT MUST HAPPEN

1 read the message

2 find the order

3 check return window

4 ask for & review photos

5 decide: refund?

6 check the £ limit

7 issue return label

8 email it to Maya

9 ⌚ WAIT UP TO 14 DAYS FOR THE PARCEL TO COME BACK – across deploys, restarts, weekends, holidays

10 parcel scanned in

11 warehouse inspects

12 refund £189 🗑️

13 restock or write off

14 file carrier damage claim

15 close the case, notify Maya

15 steps · 8 external systems · 2 weeks · real money · one irreversible action

Step 12 is the one that keeps people awake. It moves money, it can't be quietly undone, and it sits at the far end of a two-week gap where anything can have gone wrong.



Production Principles — What Actually Matters

Design long-horizon agent systems that stay correct, no matter what goes wrong.



1 Assume everything will fail

→ Models, tools, APIs, networks, workers—and humans.



2 Persist state, not just prompts

→ The workflow must know exactly where it is after a restart.



3 Separate memory from truth

→ Agent memory helps reasoning; authoritative systems determine reality.



4 Make every side effect idempotent

→ Retrying must never mean charging, refunding, emailing, or executing twice.



5 Detect → Recover → Verify

→ Never assume a successful retry means the workflow is now correct.



6 Escalate based on risk, not every exception

→ Automate routine recovery; involve humans when uncertainty or impact crosses a boundary.



7 Design for replay and explainability

→ You should be able to answer: What happened? Why? What state/data was used? How did it recover?



Design the agent assuming it will be **interrupted, confused, retried, and wrong—** and make the system safe anyway.



Key Takeaways

Designing long-horizon, self-healing agentic AI systems for finance



1

Long-horizon changes the problem

Hours → days → weeks means failures, waits, changing state, and restarts are normal.



2

Durability is the foundation

The process may die.
The workflow must not.



3

State ≠ Memory ≠ Truth

Persist state explicitly and keep authoritative truth outside the LLM.



4

Self-healing starts with detection

Detect → Diagnose → Recover → Verify
Recovery is useless if you don't know something went wrong.



5

Reliability is a system property

Don't rely on a "smarter agent." Combine durable execution, idempotency, observability, recovery, risk gates, and human oversight.



The goal isn't an agent that never fails.

It's a system that fails safely, recovers intelligently, and keeps moving forward.

